

第二周：优化算法 (Optimization algorithms)

第二周：优化算法 (Optimization algorithms)

- 2.1 Mini-batch 梯度下降 (Mini-batch gradient descent)
- 2.2 理解mini-batch梯度下降法 (Understanding mini-batch gradient descent)
- 2.3 指数加权平均数 (Exponentially weighted averages)
- 2.4 理解指数加权平均数 (Understanding exponentially weighted averages)
- 2.5 指数加权平均的偏差修正 (Bias correction in exponentially weighted averages)
- 2.6 动量梯度下降法 (Gradient descent with Momentum)
- 2.7 RMSprop
- 2.8 Adam 优化算法(Adam optimization algorithm)
- 2.9 学习率衰减(Learning rate decay)
- 2.10 局部最优的问题(The problem of local optima)

2.1 Mini-batch 梯度下降 (Mini-batch gradient descent)

本周将学习优化算法，这能让你的神经网络运行得更快。机器学习的应用是一个高度依赖经验的过程，伴随着大量迭代的过程，你需要训练诸多模型，才能找到合适的那一个，所以，优化算法能够帮助你快速训练模型。

其中一个难点在于，深度学习没有在大数据领域发挥最大的效果，我们可以利用一个巨大的数据集来训练神经网络，而在巨大的数据集基础上进行训练速度很慢。因此，你会发现，使用快速的优化算法，使用好用的优化算法能够大大提高你和团队的效率，那么，我们首先来谈谈**mini-batch**梯度下降法。

Batch vs. mini-batch gradient descent

Vectorization allows you to efficiently compute on m examples.

$$X = \begin{bmatrix} x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(500)} & | & x^{(501)} & \dots & x^{(1000)} & | & \dots & | & \dots & x^{(m)} \end{bmatrix}$$

(n_x, m) $\underbrace{\hspace{10em}}_{X^{[1:]}}$ $\underbrace{\hspace{10em}}_{X^{[2:]}}$ $\dots$ $\underbrace{\hspace{10em}}_{X^{[5,000]}}$

$$Y = \begin{bmatrix} y^{(1)} & y^{(2)} & y^{(3)} & \dots & y^{(500)} & | & y^{(501)} & \dots & y^{(1000)} & | & \dots & | & \dots & y^{(m)} \end{bmatrix}$$

$(1, m)$ $\underbrace{\hspace{10em}}_{Y^{[1:]}}$ $\underbrace{\hspace{10em}}_{Y^{[2:]}}$ $\dots$ $\underbrace{\hspace{10em}}_{Y^{[5,000]}}$

What if $m = 5,000,000$?

5,000 mini-batches of 1,000 each

Mini-batch t : $X^{[t:5]}, Y^{[t:5]}$

$x^{(i)}$
 $z^{[2]}$
 σ

Andrew Ng

你之前学过，向量化能够让你有效地对所有 m 个样本进行计算，允许你处理整个训练集，而无需某个明确的公式。所以我们要把训练样本放大巨大的矩阵 X 当中去， $X = [x^{(1)} \ x^{(2)} \ x^{(3)} \ \dots \ x^{(m)}]$ ， Y 也是如此， $Y = [y^{(1)} \ y^{(2)} \ y^{(3)} \ \dots \ y^{(m)}]$ ，所以 X 的维数是 (n_x, m) ， Y 的维数是 $(1, m)$ ，向量化能够让你相对较快地处理所有 m 个样本。如果 m 很大的话，处理速度仍然缓慢。比如说，如果 m 是500万或5000万或者更大的一个数，在对整个训练集执行梯度下降法时，你要做的是，你必须处理整个训练集，然后才能进行一步梯度下降法，然后你需要再

重新处理500万个训练样本，才能进行下一步梯度下降法。所以如果你在处理完整个500万个样本的训练集之前，先让梯度下降法处理一部分，你的算法速度会更快，准确地说，这是你可以做的一些事情。

你可以把训练集分割为小一点的子集训练，这些子集被取名为**mini-batch**，假设每一个子集中只有1000个样本，那么把其中的 $x^{(1)}$ 到 $x^{(1000)}$ 取出来，将其称为第一个子训练集，也叫做**mini-batch**，然后你再取出接下来的1000个样本，从 $x^{(1001)}$ 到 $x^{(2000)}$ ，然后再取1000个样本，以此类推。

接下来我要说一个新的符号，把 $x^{(1)}$ 到 $x^{(1000)}$ 称为 $X^{\{1\}}$ ， $x^{(1001)}$ 到 $x^{(2000)}$ 称为 $X^{\{2\}}$ ，如果你的训练样本一共有500万个，每个**mini-batch**都有1000个样本，也就是说，你有5000个**mini-batch**，因为5000乘以1000就是5000万。

What if $m = 5,000,000$?
5,000 mini-batches of 1,000 each

你共有5000个**mini-batch**，所以最后得到是 $X^{\{5000\}}$

$$X = \underbrace{\begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(1000)} \end{bmatrix}}_{X^{\{1\}}} \underbrace{\begin{bmatrix} x^{(1001)} & \dots & x^{(2000)} \end{bmatrix}}_{X^{\{2\}}} \dots \underbrace{\begin{bmatrix} \dots & x^{(m)} \end{bmatrix}}_{X^{\{5000\}}}$$

(n_x, m)

对 Y 也要进行相同处理，你也要相应地拆分 Y 的训练集，所以这是 $Y^{\{1\}}$ ，然后从 $y^{(1001)}$ 到 $y^{(2000)}$ ，这个叫 $Y^{\{2\}}$ ，一直到 $Y^{\{5000\}}$ 。

$$Y = \underbrace{\begin{bmatrix} y^{(1)} & y^{(2)} & \dots & y^{(1000)} \end{bmatrix}}_{Y^{\{1\}}} \underbrace{\begin{bmatrix} y^{(1001)} & \dots & y^{(2000)} \end{bmatrix}}_{Y^{\{2\}}} \dots \underbrace{\begin{bmatrix} \dots & y^{(m)} \end{bmatrix}}_{Y^{\{5000\}}}$$

$(1, m)$

mini-batch的数量 t 组成了 $X^{\{t\}}$ 和 $Y^{\{t\}}$ ，这就是1000个训练样本，包含相应的输入输出对。

What if $m = 5,000,000$?
5,000 mini-batches of 1,000 each
Mini-batch t : $X^{\{t\}}, Y^{\{t\}}$

$$\left| \begin{array}{l} x^{(i)} \\ z^{[l]} \\ X^{\{t\}}, Y^{\{t\}} \end{array} \right.$$

在继续课程之前，先确定一下我的符号，之前我们使用了上角小括号 (i) 表示训练集里的值，所以 $x^{(i)}$ 是第 i 个训练样本。我们用了上角中括号 $[l]$ 来表示神经网络的层数， $z^{[l]}$ 表示神经网络中第 l 层的 z 值，我们现在引入了大括号 t 来代表不同的**mini-batch**，所以我们有 $X^{\{t\}}$ 和 $Y^{\{t\}}$ ，检查一下自己是否理解无误。

$$X = \underbrace{\begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(1000)} \end{bmatrix}}_{X^{\{1\}} (n_x, 1000)} \underbrace{\begin{bmatrix} x^{(1001)} & \dots & x^{(2000)} \end{bmatrix}}_{X^{\{2\}} (n_x, 1000)} \dots \underbrace{\begin{bmatrix} \dots & x^{(m)} \end{bmatrix}}_{X^{\{5000\}} (n_x, 1000)}$$

(n_x, m)

$$Y = \underbrace{\begin{bmatrix} y^{(1)} & y^{(2)} & \dots & y^{(1000)} \end{bmatrix}}_{Y^{\{1\}} (1, 1000)} \underbrace{\begin{bmatrix} y^{(1001)} & \dots & y^{(2000)} \end{bmatrix}}_{Y^{\{2\}} (1, 1000)} \dots \underbrace{\begin{bmatrix} \dots & y^{(m)} \end{bmatrix}}_{Y^{\{5000\}} (1, 1000)}$$

$(1, m)$

$X^{\{t\}}$ 和 $Y^{\{t\}}$ 的维数：如果 $X^{\{1\}}$ 是一个有1000个样本的训练集，或者说是1000个样本的 x 值，所以维数应该是 $(n_x, 1000)$ ， $X^{\{2\}}$ 的维数应该是 $(n_x, 1000)$ ，以此类推。因此所有的子集维数都是 $(n_x, 1000)$ ，而这些 $(Y^{\{t\}})$ 的维数都是 $(1, 1000)$ 。

解释一下这个算法的名称，**batch**梯度下降法指的是我们之前讲过的梯度下降法算法，就是同时处理整个训练集，这个名字就是来源于能够同时看到整个**batch**训练集的样本被处理，这个名字不怎么样，但就是这样叫它。

相比之下，**mini-batch**梯度下降法，指的是我们在下一张幻灯片中会讲到的算法，你每次同时处理的单个的**mini-batch** $X^{\{t\}}$ 和 $Y^{\{t\}}$ ，而不是同时处理全部的 X 和 Y 训练集。

那么究竟**mini-batch**梯度下降法的原理是什么？在训练集上运行**mini-batch**梯度下降法，你运行 `for t=1.....5000`，因为我们有5000个各有1000个样本的组，在**for**循环里你要做得基本就是对 $X^{\{t\}}$ 和 $Y^{\{t\}}$ 执行一步梯度下降法。假设你有一个拥有1000个样本的训练集，而且假设你已经很熟悉一次性处理完的方法，你要用向量化去几乎同时处理1000个样本。

Mini-batch gradient descent

repeat $\{$ for $t = 1, \dots, 5000 \{$

Forward prop on $X^{\{t\}}$.

$$z^{(t)} = W^{(t)} X^{\{t\}} + b^{(t)}$$

$$A^{(t)} = g^{(t)}(z^{(t)})$$

Vectorized implementation (1000 examples)

Compute cost $J^{(t)} = \frac{1}{1000} \sum_{i=1}^{1000} L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_l \|W^{(t)}\|_F^2$

Backprop to compute gradients w.r.t $J^{(t)}$ (using $X^{\{t\}}, Y^{\{t\}}$)

$$W^{(t+1)} = W^{(t)} - \alpha dW^{(t)}, b^{(t+1)} = b^{(t)} - \alpha db^{(t)}$$

"1 epoch" pass through training set.

Andrew Ng

首先对输入也就是 $X^{\{t\}}$ ，执行前向传播，然后执行 $z^{[1]} = W^{[1]} X + b^{[1]}$ ，之前我们这里只有，但是现在你正在处理整个训练集，你在处理第一个**mini-batch**，在处理**mini-batch**时它变成了 $X^{\{t\}}$ ，即 $z^{[1]} = W^{[1]} X^{\{t\}} + b^{[1]}$ ，然后执行 $A^{[1]k} = g^{[1]}(z^{[1]})$ ，之所以用大写的 Z 是因为这是一个向量内涵，以此类推，直到 $A^{[L]} = g^{[L]}(Z^{[L]})$ ，这就是你的预测值。注意这里你需要用到一个向量化的执行命令，这个向量化的执行命令，一次性处理1000个而不是500万个样本。接下来你要计算损失成本函数 J ，因为子集规模是1000， $J = \frac{1}{1000} \sum_{i=1}^l L(\hat{y}^{(i)}, y^{(i)})$ ，说明一下，这 $(L(\hat{y}^{(i)}, y^{(i)}))$ 指的是来自于**mini-batch** $X^{\{t\}}$ 和 $Y^{\{t\}}$ 中的样本。

如果你用到了正则化，你也可以使用正则化的术语， $J = \frac{1}{1000} \sum_{i=1}^l L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_l \|w^{[l]}\|_F^2$ ，因为这是一个**mini-batch**的损失，所以我将 J 损失记为上角标 t ，放在大括号里（

$$J^{\{t\}} = \frac{1}{1000} \sum_{i=1}^l L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_l \|w^{[l]}\|_F^2。$$

你也会注意到，我们做的一切似曾相识，其实跟之前我们执行梯度下降法如出一辙，除了你现在的对象不是 X, Y ，而是 $X^{\{t\}}$ 和 $Y^{\{t\}}$ 。接下来，你执行反向传播来计算 $J^{\{t\}}$ 的梯度，你只是使用 $X^{\{t\}}$ 和 $Y^{\{t\}}$ ，然后你更新加权值， W 实际上是 $W^{[l]}$ ，更新为 $W^{[l]} := W^{[l]} - \alpha dW^{[l]}$ ，对 b 做相同处理， $b^{[l]} := b^{[l]} - \alpha db^{[l]}$ 。这是使用**mini-batch**梯度下降法训练样本的一步，我写下的代码也可被称为进行“一代”(1 epoch)的训练。一代这个词意味着只是一次遍历了训练集。

Mini-batch gradient descent

repeat $\{$
 for $t = 1, \dots, 5000 \{$

Forward prop on $X^{(t)}$.

$$Z^{(t)} = W^{(t)} X^{(t)} + b^{(t)}$$

$$A^{(t)} = \sigma^{(t)}(Z^{(t)})$$

$\vdots$

$$A^{(t)} = \sigma^{(t)}(Z^{(t)})$$

Batched inference (1000 examples)

Compute cost $J = \frac{1}{1000} \sum_{i=1}^{1000} \ell(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_{i=1}^{1000} \|W^{(t)}\|_F^2$

Backprop to compute gradients w.r.t $J^{(t)}$ (using $X^{(t)}, Y^{(t)}$)

$$W^{(t+1)} = W^{(t)} - \alpha \frac{\partial J}{\partial W}, \quad b^{(t+1)} = b^{(t)} - \alpha \frac{\partial J}{\partial b}$$

$\} \}$

"1 epoch" $\leftarrow$ pass through training set.

1 step of gradient descent using $X^{(t+1)}, Y^{(t+1)}$ (as if $t=1000$)

X, Y

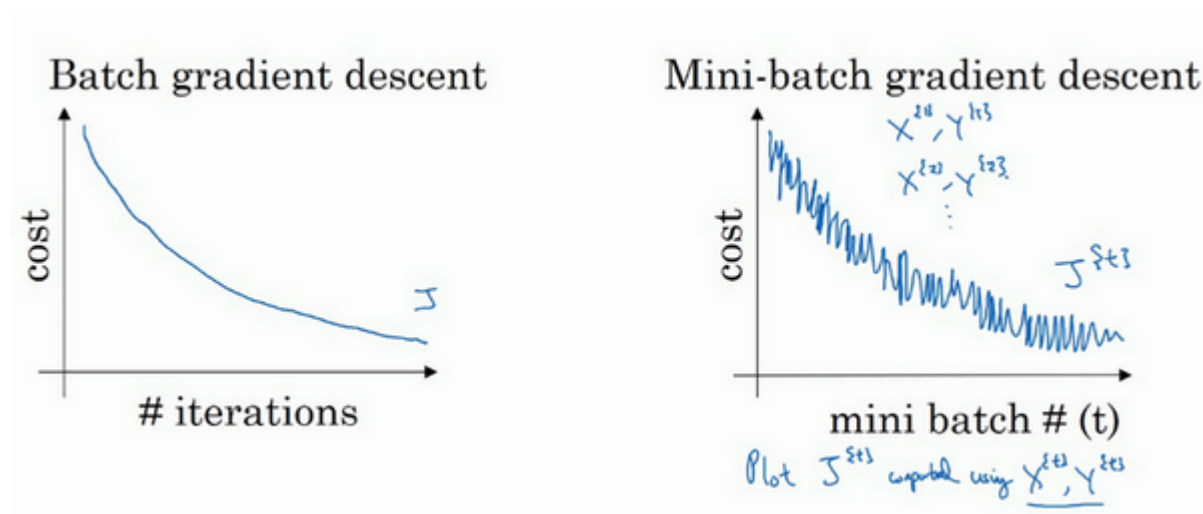
Andrew Ng

使用**batch**梯度下降法，一次遍历训练集只能让你做一个梯度下降，使用**mini-batch**梯度下降法，一次遍历训练集，能让你做5000个梯度下降。当然正常来说你想要多次遍历训练集，还需要为另一个**while**循环设置另一个**for**循环。所以你可以一直处理遍历训练集，直到最后你能收敛到一个合适的精度。

如果你有一个丢失的训练集，**mini-batch**梯度下降法比**batch**梯度下降法运行地更快，所以几乎每个研习深度学习的人在训练巨大的数据集时都会用到，下一个视频中，我们将进一步深度讨论**mini-batch**梯度下降法，你也会因此更好地理解它的作用和原理。

2.2 理解mini-batch梯度下降法 (Understanding mini-batch gradient descent)

在上周视频中，你知道了如何利用**mini-batch**梯度下降法来开始处理训练集和开始梯度下降，即使你只处理了部分训练集，即使你是第一次处理，本视频中，我们将进一步学习如何执行梯度下降法，更好地理解其作用和原理。



使用**batch**梯度下降法时，每次迭代你都需要遍历整个训练集，可以预期每次迭代成本都会下降，所以如果成本函数 J 是迭代次数的一个函数，它应该会随着每次迭代而减少，如果 J 在某次迭代中增加了，那肯定出了问题，也许你的学习率太大。

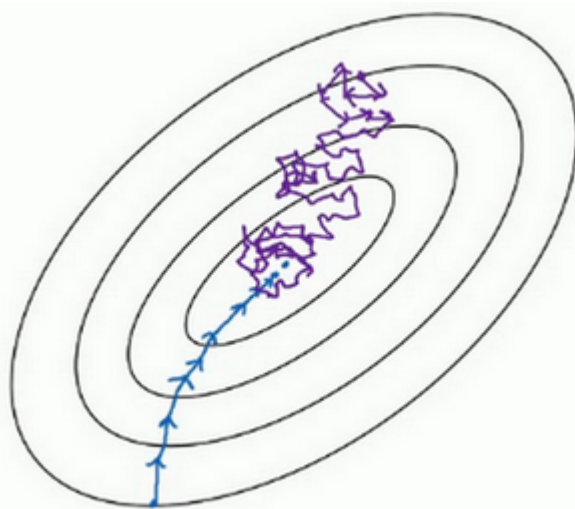
使用**mini-batch**梯度下降法，如果你作出成本函数在整个过程中的图，则并不是每次迭代都是下降的，特别是在每次迭代中，你要处理的是 $X^{\{t\}}$ 和 $Y^{\{t\}}$ ，如果要作出成本函数 $J^{\{t\}}$ 的图，而 $J^{\{t\}}$ 只和 $X^{\{t\}}$ ， $Y^{\{t\}}$ 有关，也就是每次迭代下你都在训练不同的样本集或者说训练不同的**mini-batch**，如果你要作出成本函数 J 的图，你很可能会看到这样的结果，走向朝下，但有更多的噪声，所以如果你作出 $J^{\{t\}}$ 的图，因为在训练**mini-batch**梯度下降法时，会经过多代，你可能会看到这样的曲线。没有每次迭代都下降是不要紧的，但走势应该向下，噪声产生的原因在于也许 $X^{\{1\}}$ 和 $Y^{\{1\}}$ 是比较容易计算的**mini-batch**，因此成本会低一些。不过也许出于偶然， $X^{\{2\}}$ 和 $Y^{\{2\}}$ 是比较难运算的**mini-batch**，或许你需要一些残缺的样本，这样一来，成本会更高一些，所以才会出现这些摆动，因为你是运行**mini-batch**梯度下降法作出成本函数图。

你需要决定的变量之一是**mini-batch**的大小， m 就是训练集的大小，极端情况下，如果**mini-batch**的大小等于 m ，其实就是**batch**梯度下降法，在这种极端情况下，你就有了**mini-batch** $X^{\{1\}}$ 和 $Y^{\{1\}}$ ，并且该**mini-batch**等于整个训练集，所以把**mini-batch**大小设为 m 可以得到**batch**梯度下降法。

Choosing your mini-batch size

→ If mini-batch size = m : Batch gradient descent. $(X^{\{1\}}, Y^{\{1\}}) = (X, Y)$
 → If mini-batch size = 1 : Stochastic gradient descent. Every example is its own $(X^{\{1\}}, Y^{\{1\}}) = (x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$ mini-batch.

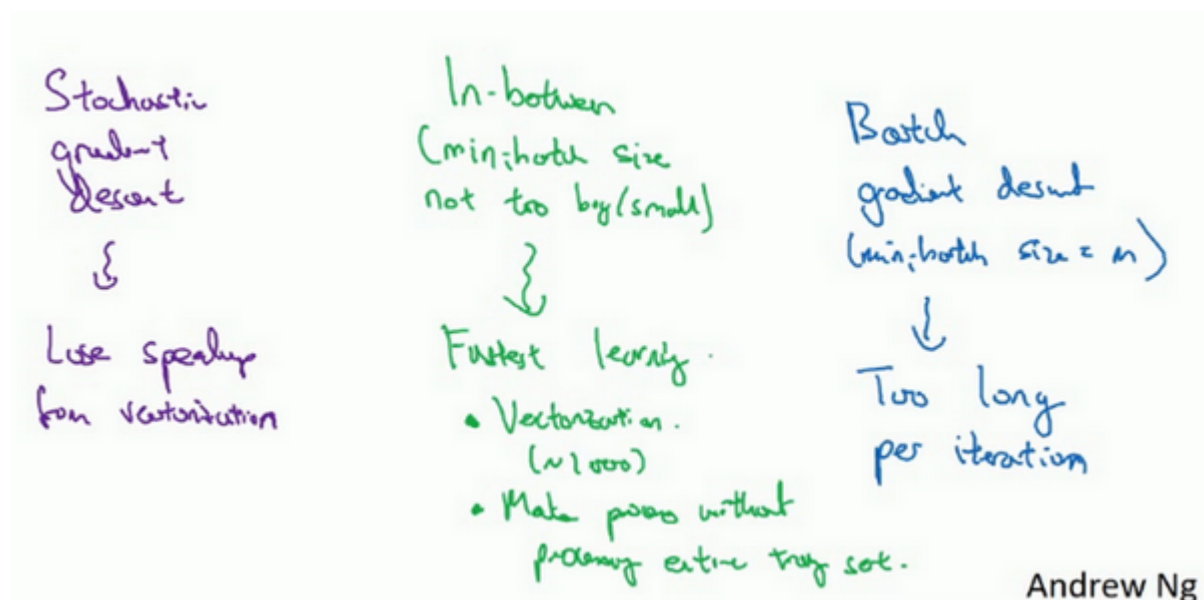
另一个极端情况，假设**mini-batch**大小为1，就有了新的算法，叫做随机梯度下降法，每个样本都是独立的**mini-batch**，当你看第一个**mini-batch**，也就是 $X^{\{1\}}$ 和 $Y^{\{1\}}$ ，如果**mini-batch**大小为1，它就是你的第一个训练样本，这就是你的第一个训练样本。接着再看第二个**mini-batch**，也就是第二个训练样本，采取梯度下降步骤，然后是第三个训练样本，以此类推，一次只处理一个。



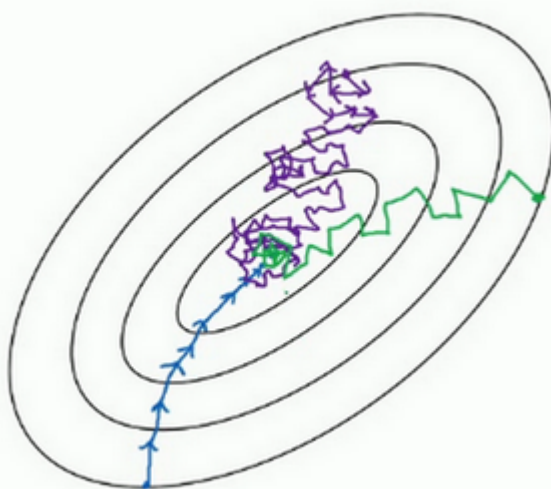
看在两种极端下成本函数的优化情况，如果这是你想要最小化的成本函数的轮廓，最小值在那里，**batch**梯度下降法从某处开始，相对噪声低些，幅度也大一些，你可以继续找最小值。

相反，在随机梯度下降法中，从某一点开始，我们重新选取一个起始点，每次迭代，你只对一个样本进行梯度下降，大部分时候你向着全局最小值靠近，有时候你会远离最小值，因为那个样本恰好给你指的方向不对，因此随机梯度下降法是有噪声的，平均来看，它最终会靠近最小值，不过有时候也会方向错误，因为随机梯度下降法永远不会收敛，而是会一直在最小值附近波动，但它并不会在达到最小值并停留在此。

实际上你选择的**mini-batch**大小在二者之间，大小在1和 m 之间，而1太小了， m 太大了，原因在于如果使用**batch**梯度下降法，**mini-batch**的大小为 m ，每个迭代需要处理大量训练样本，该算法的主要弊端在于特别是在训练样本数量巨大的时候，单次迭代耗时太长。如果训练样本不大，**batch**梯度下降法运行地很好。



相反，如果使用随机梯度下降法，如果你只要处理一个样本，那这个方法很好，这样做没有问题，通过减小学习率，噪声会被改善或有所减小，但随机梯度下降法的一大缺点是，你会失去所有向量化带给你的加速，因为一次性只处理了一个训练样本，这样效率过于低下，所以实践中最好选择不大不小的**mini-batch**尺寸，实际上学习率达到最快。你会发现两个好处，一方面，你得到了大量向量化，上个视频中我们用过的例子中，如果**mini-batch**大小为1000个样本，你就可以对1000个样本向量化，比你一次性处理多个样本快得多。另一方面，你不需要等待整个训练集被处理完就可以开始进行后续工作，再用一下上个视频的数字，每次训练集允许我们采取5000个梯度下降步骤，所以实际上一些位于中间的**mini-batch**大小效果最好。



用**mini-batch**梯度下降法，我们从这里开始，一次迭代这样做，两次，三次，四次，它不会总朝向最小值靠近，但它比随机梯度下降要更持续地靠近最小值的方向，它也不一定在很小的范围内收敛或者波动，如果出现这个问题，可以慢慢减少学习率，我们在下个视频会讲到学习率衰减，也就是如何减小学习率。

如果**mini-batch**大小既不是1也不是 m ，应该取中间值，那应该怎么选择呢？其实是有指导原则的。

首先，如果训练集较小，直接使用**batch**梯度下降法，样本集较小就没必要使用**mini-batch**梯度下降法，你可以快速处理整个训练集，所以使用**batch**梯度下降法也很好，这里的少是说小于2000个样本，这样比较适合使用**batch**梯度下降法。不然，样本数目较大的话，一般的**mini-batch**大小为64到512，考虑到电脑内存设置和使用的方式，如果**mini-batch**大小是2的 n 次方，代码会运行地快一些，64就是2的6次方，以此类推，128是2的7次方，256是2的8次方，512是2的9次方。所以我经常把**mini-batch**大小设成2的次方。在上一个视频里，我的**mini-batch**大小

设为了1000，建议你可以试一下1024，也就是2的10次方。也有**mini-batch**的大小为1024，不过比较少见，64到512的**mini-batch**比较常见。

最后需要注意的是在你的**mini-batch**中，要确保 $X^{\{t\}}$ 和 $Y^{\{t\}}$ 要符合CPU/GPU内存，取决于你的应用方向以及训练集的大小。如果你处理的**mini-batch**和CPU/GPU内存不相符，不管你用什么方法处理数据，你会注意到算法的表现急转直下变得惨不忍睹，所以我希望你对一般人们使用的**mini-batch**大小有一个直观了解。事实上**mini-batch**大小是另一个重要的变量，你需要做一个快速尝试，才能找到能够最有效地减少成本函数的那个，我一般会尝试几个不同的值，几个不同的2次方，然后看能否找到一个让梯度下降优化算法最高效的大小。希望这些能够指导你如何开始找到这一数值。

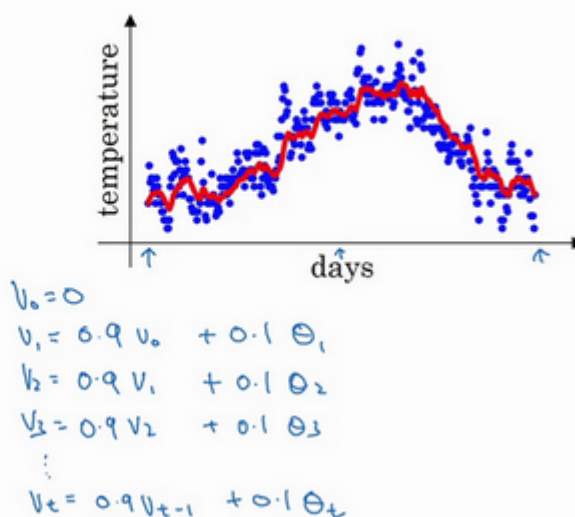
你学会了如何执行**mini-batch**梯度下降，令算法运行得更快，特别是在训练样本数目较大的情况下。不过还有个更高效的算法，比梯度下降法和**mini-batch**梯度下降法都要高效的多，我们在接下来的视频中将为大家一一讲解。

2.3 指数加权平均数 (Exponentially weighted averages)

我想向你展示几个优化算法，它们比梯度下降法快，要理解这些算法，你需要用到指数加权平均，在统计中也叫做指数加权移动平均，我们首先讲这个，然后再来讲更复杂的优化算法。

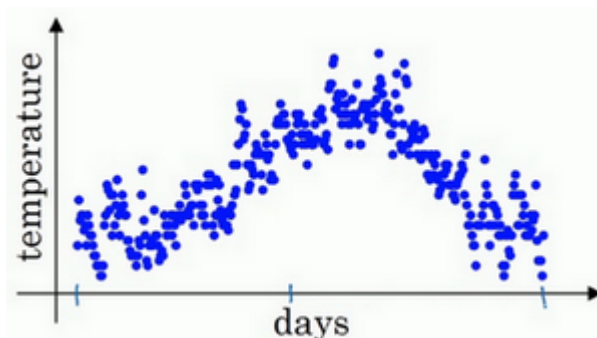
Temperature in London

$\theta_1 = 40^\circ\text{F}$ 4°C
 $\theta_2 = 49^\circ\text{F}$ 9°C
 $\theta_3 = 45^\circ\text{F}$...
 $\vdots$
 $\theta_{180} = 60^\circ\text{F}$ 15°C
 $\theta_{181} = 56^\circ\text{F}$...
 $\vdots$

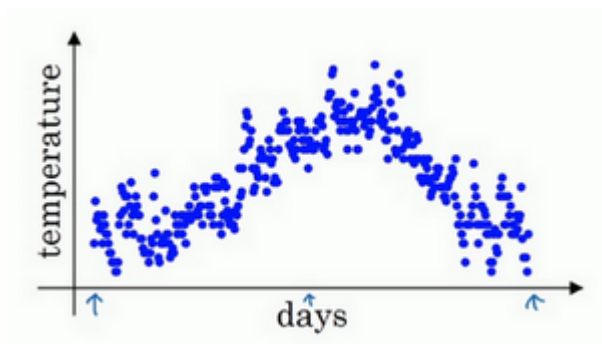


Andrew Ng

虽然现在我生活在美国，实际上我生于英国伦敦。比如我这儿有去年伦敦的每日温度，所以1月1号，温度是40华氏度，相当于4摄氏度。我知道世界上大部分地区使用摄氏度，但是美国使用华氏度。在1月2号是9摄氏度等等。在年中的时候，一年365天，年中就是说，大概180天的样子，也就是5月末，温度是60华氏度，也就是15摄氏度等等。夏季温度转暖，然后冬季降温。



你用数据作图，可以得到以下结果，起始日在1月份，这里是夏季初，这里是年末，相当于12月末。



这里是1月1号，年中接近夏季的时候，随后就是年末的数据，看起来有些杂乱，如果要计算趋势的话，也就是温度的局部平均值，或者说移动平均值。

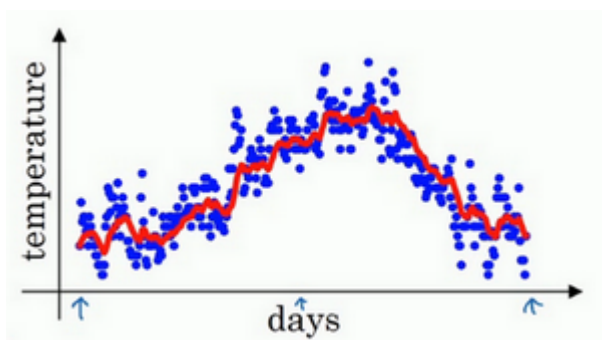
$$\begin{aligned}
 v_0 &= 0 \\
 v_1 &= 0.9 v_0 + 0.1 \theta_1 \\
 v_2 &= 0.9 v_1 + 0.1 \theta_2 \\
 v_3 &= 0.9 v_2 + 0.1 \theta_3 \\
 &\vdots \\
 v_t &= 0.9 v_{t-1} + 0.1 \theta_t
 \end{aligned}$$

你要做的是，首先使 $v_0 = 0$ ，每天，需要使用0.9的加权数之前的数值加上当日温度的0.1倍，即 $v_1 = 0.9v_0 + 0.1\theta_1$ ，所以这里是第一天的温度值。

第二天，又可以获得一个加权平均数，0.9乘以之前的值加上当日的温度0.1倍，即 $v_2 = 0.9v_1 + 0.1\theta_2$ ，以此类推。

第二天值加上第三日数据的0.1，如此往下。大体公式就是某天的 v 等于前一天 v 值的0.9加上当日温度的0.1。

如此计算，然后用红线作图的话，便得到这样的结果。



你得到了移动平均值，每日温度的指数加权平均值。

看一下上一张幻灯片里的公式， $v_t = 0.9v_{t-1} + 0.1\theta_t$ ，我们把0.9这个常数变成 β ，将之前的0.1变成 $(1 - \beta)$ ，即 $v_t = \beta v_{t-1} + (1 - \beta)\theta_t$

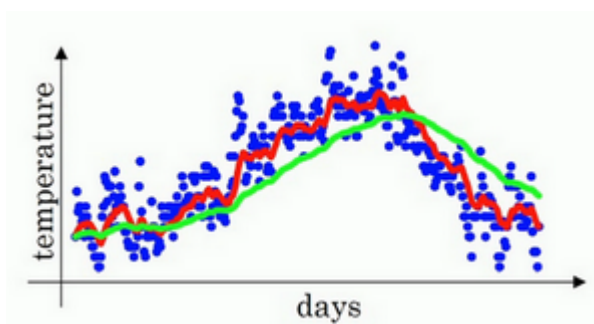
$$v_t = \beta v_{t-1} + (1-\beta) \theta_t$$

$\beta = 0.9$: ≈ 10 days' temperature.
 $\beta = 0.98$: ≈ 50 days

v_t is approximately
 average over
 $\approx \frac{1}{1-\beta}$ days' temperature.

由于以后我们要考虑的原因，在计算时可视 v_t 大概是 $\frac{1}{(1-\beta)}$ 的每日温度，如果 β 是0.9，你会想，这是十天的平均值，也就是红线部分。

我们来试试别的，将 β 设置为接近1的一个值，比如0.98，计算 $\frac{1}{(1-0.98)} = 50$ ，这就是粗略平均了一下，过去50天的温度，这时作图可以得到绿线。

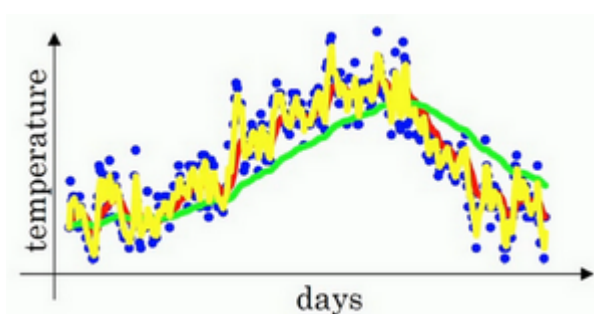


这个高值 β 要注意几点，你得到的曲线要平坦一些，原因在于你多平均了几天的温度，所以这个曲线，波动更小，更加平坦，缺点是曲线进一步右移，因为现在平均的温度值更多，要平均更多的值，指数加权平均公式在温度变化时，适应地更缓慢一些，所以会出现一定延迟，因为当 $\beta = 0.98$ ，相当于给前一天的值加了太多权重，只有0.02的权重给了当日的值，所以温度变化时，温度上下起伏，当 β 较大时，指数加权平均值适应地更缓慢一些。

我们可以再换一个值试一试，如果 β 是另一个极端值，比如说0.5，根据右边的公式 $(\frac{1}{(1-\beta)})$ ，这是平均了两天的温度。

$$\beta = 0.5 : \approx 2 \text{ days}$$

作图运行后得到黄线。



由于仅平均了两天的温度，平均的数据太少，所以得到的曲线有更多的噪声，有可能出现异常值，但是这个曲线能够更快适应温度变化。

所以指数加权平均数经常被使用，再说一次，它在统计学中被称为指数加权移动平均值，我们就简称为指数加权平均数。通过调整这个参数 (β) ，或者说后面的算法学习，你会发现这是一个很重要的参数，可以取得稍微不同的效果，往往中间有某个值效果最好， β 为中间值时得到的红色曲线，比起绿线和黄线更好地平均了温度。

现在你知道计算指数加权平均数的基本原理，下一个视频中，我们再聊聊它的本质作用。

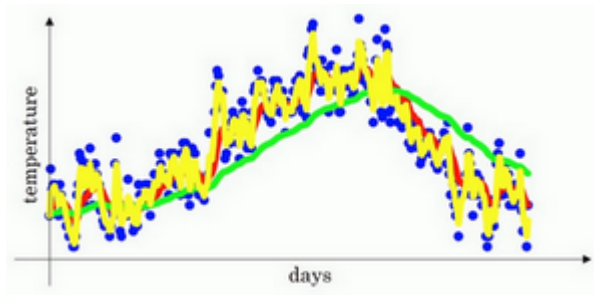
2.4 理解指数加权平均数 (Understanding exponentially weighted averages)

上个视频中，我们讲到了指数加权平均数，这是几个优化算法中的关键一环，而这几个优化算法能帮助你训练神经网络。本视频中，我希望进一步探讨算法的本质作用。

回忆一下这个计算指数加权平均数的关键方程。

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

$\beta = 0.9$ 的时候，得到的结果是红线，如果它更接近于1，比如0.98，结果就是绿线，如果 β 小一点，如果是0.5，结果就是黄线。



我们进一步地分析，来理解如何计算出每日温度的平均值。

同样的公式， $v_t = \beta v_{t-1} + (1 - \beta)\theta_t$

使 $\beta = 0.9$ ，写下相应的几个公式，所以在执行的时候， t 从0到1到2到3， t 的值在不断增加，为了更好地分析，我写的时候使得 t 的值不断减小，然后继续往下写。

$$\begin{aligned} v_{100} &= 0.9v_{99} + 0.1\theta_{100} \\ v_{99} &= 0.9v_{98} + 0.1\theta_{99} \\ v_{98} &= 0.9v_{97} + 0.1\theta_{98} \\ &\dots \end{aligned}$$

首先看第一个公式，理解 v_{100} 是什么？我们调换一下这两项（ $0.9v_{99} + 0.1\theta_{100}$ ）， $v_{100} = 0.1\theta_{100} + 0.9v_{99}$ 。

那么 v_{99} 是什么？我们就代入这个公式（ $v_{99} = 0.1\theta_{99} + 0.9v_{98}$ ），所以：

$$v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98})$$

那么 v_{98} 是什么？你可以用这个公式计算（ $v_{98} = 0.1\theta_{98} + 0.9v_{97}$ ），把公式代进去，所以：

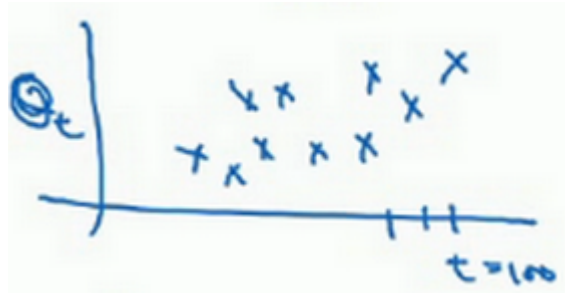
$$v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9v_{97}))$$

以此类推，如果你把这些括号都展开，

$$v_{100} = 0.1\theta_{100} + 0.1 \times 0.9\theta_{99} + 0.1 \times (0.9)^2\theta_{98} + 0.1 \times (0.9)^3\theta_{97} + 0.1 \times (0.9)^4\theta_{96} + \dots$$

$$\begin{aligned} V_{100} &= 0.1 \theta_{100} + 0.9 \cancel{\theta_{99}} (0.1 \theta_{99} + 0.9 \cancel{\theta_{98}}) \quad \begin{array}{l} \uparrow \quad \quad \quad \uparrow \\ 0.1 \theta_{97} + 0.9 \theta_{96} \end{array} \\ &= 0.1 \theta_{100} + 0.1 \times 0.9 \cdot \theta_{99} + 0.1 (0.9)^2 \theta_{97} + 0.1 (0.9)^2 \theta_{96} + 0.1 (0.9)^4 \theta_{94} + \dots \end{aligned}$$

所以这是一个加和并平均，100号数据，也就是当日温度。我们分析 v_{100} 的组成，也就是在一年第100天计算的数据，但是这个总和，包括100号数据，99号数据，97号数据等等。画图的一个办法是，假设我们有一些日期的温度，所以这是数据，这是 t ，所以100号数据有个数值，99号数据有个数值，98号数据等等， t 为100，99，98等等，这就是数日的温度数值。



然后我们构建一个指数衰减函数，从0.1开始，到 0.1×0.9 ，到 $0.1 \times (0.9)^2$ ，以此类推，所以就有了这个指数衰减函数。



计算 v_{100} 是通过，把两个函数对应的元素，然后求和，用这个数值100号数据值乘以0.1，99号数据值乘以0.1乘以 $(0.9)^2$ ，这是第二项，以此类推，所以选取的是每日温度，将其与指数衰减函数相乘，然后求和，就得到了 v_{100} 。



结果是，稍后我们详细讲解，不过所有的这些系数 ($0.1 \cdot 0.1 \times 0.9 \cdot 0.1 \times (0.9)^2 \cdot 0.1 \times (0.9)^3 \dots$)，相加起来为1或者逼近1，我们称之为偏差修正，下个视频会涉及。

最后也许你会问，到底需要平均多少天的温度。实际上 $(0.9)^{10}$ 大约为0.35，这大约是 $\frac{1}{e}$ ， e 是自然算法的基础之一。大体上说，如果有 $1 - \varepsilon$ ，在这个例子中， $\varepsilon = 0.1$ ，所以 $1 - \varepsilon = 0.9$ ， $(1 - \varepsilon)^{\frac{1}{\varepsilon}}$ 约等于 $\frac{1}{e}$ ，大约是0.34，0.35，换句话说，10天后，曲线的高度下降到 $\frac{1}{3}$ ，相当于在峰值的 $\frac{1}{e}$ 。

$$1 - \varepsilon$$

又因此当 $\beta = 0.9$ 的时候，我们说仿佛你在计算一个指数加权平均数，只关注了过去10天的温度，因为10天后，权重下降到不到当日权重的三分之一。

相反，如果，那么0.98需要多少次方才能达到这么小的数值？ $(0.98)^{50}$ 大约等于 $\frac{1}{e}$ ，所以前50天这个数值比 $\frac{1}{e}$ 大，数值会快速衰减，所以本质上这是一个下降幅度很大的函数，你可以看作平均了50天的温度。因为在例子中，要代入等式的左边， $\varepsilon = 0.02$ ，所以 $\frac{1}{\varepsilon}$ 为50，我们由此得到公式，我们平均了大约 $\frac{1}{(1-\beta)}$ 天的温度，这里 ε 代替了 $1 - \beta$ ，也就是说根据一些常数，你能大概知道能够平均多少日的温度，不过这只是思考的大致方向，并不是正式的数学证明。

$$v_0 = 0$$

$$v_1 = \beta v_0 + (1 - \beta) \theta_1$$

最后讲讲如何在实际中执行，还记得吗？我们一开始将 v_0 设置为0，然后计算第一天 v_1 ，然后 v_2 ，以此类推。

现在解释一下算法，可以将 v_0, v_1, v_2 等等写成明确的变量，不过在实际中执行的话，你要做的是，一开始将 v 初始化为0，然后在第一天使 $v := \beta v + (1 - \beta) \theta_1$ ，然后第二天，更新 v 值， $v := \beta v + (1 - \beta) \theta_2$ ，以此类推，有些人会把 v 加下标，来表示 v 是用来计算数据的指数加权平均数。

$$V_{\theta} := 0$$

$$V_t := \beta V + (1 - \beta)\theta_t$$

再说一次，但是换个说法， $v_{\theta} = 0$ ，然后每一天，拿到第 t 天的数据，把 v 更新为 $v := \beta v_{\theta} + (1 - \beta)\theta_t$ 。

指数加权平均数公式的好处之一在于，它占用极少内存，电脑内存中只占用一行数字而已，然后把最新数据代入公式，不断覆盖就可以了，正因为这个原因，其效率，它基本上只占用一行代码，计算指数加权平均数也只占用单行数字的存储和内存，当然它并不是最好的，也不是最精准的计算平均数的方法。如果你要计算移动窗，你直接算出过去10天的总和，过去50天的总和，除以10和50就好，如此往往会得到更好的估测。但缺点是，如果保存所有最近的温度数据，和过去10天的总和，必须占用更多的内存，执行更加复杂，计算成本也更加高昂。

所以在接下来的视频中，我们会计算多个变量的平均值，从计算和内存效率来说，这是一个有效的方法，所以在机器学习中会经常使用，更不用说只要一行代码，这也是一个优势。

现在你学会了计算指数加权平均数，你还需要知道一个专业概念，叫做偏差修正，下一个视频我们会讲到它，接着你就可以用它构建更好的优化算法，而不是简单直接的梯度下降法。

2.5 指数加权平均的偏差修正（Bias correction in exponentially weighted averages）

你学过了如何计算指数加权平均数，有一个技术名词叫做偏差修正，可以让平均数运算更加准确，来看看它是如何运行的。

Bias correction

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

在上一个视频中，这个（红色）曲线对应 β 的值为0.9，这个（绿色）曲线对应的 $\beta=0.98$ ，如果你执行写在这里的公式，在 β 等于0.98的时候，得到的并不是绿色曲线，而是紫色曲线，你可以注意到紫色曲线的起点较低，我们来看看怎么处理。

计算移动平均数的时候，初始化 $v_0 = 0$ ， $v_1 = 0.98v_0 + 0.02\theta_1$ ，但是 $v_0 = 0$ ，所以这部分没有了（ $0.98v_0$ ），所以 $v_1 = 0.02\theta_1$ ，所以如果一天温度是40华氏度，那么 $v_1 = 0.02\theta_1 = 0.02 \times 40 = 8$ ，因此得到的值会小很多，所以第一天温度的估测不准。

$v_2 = 0.98v_1 + 0.02\theta_2$ ，如果代入 v_1 ，然后相乘，所以 $v_2 = 0.98 \times 0.02\theta_1 + 0.02\theta_2 = 0.0196\theta_1 + 0.02\theta_2$ ，假设 θ_1 和 θ_2 都是正数，计算后 v_2 要远小于 θ_1 和 θ_2 ，所以 v_2 不能很好估测出这一年前两天的温度。

有个办法可以修改这一估测，让估测变得更好，更准确，特别是在估测初期，也就是不用 v_t ，而是用 $\frac{v_t}{1-\beta^t}$ ， t 就是现在的天数。举个具体例子，当 $t = 2$ 时， $1 - \beta^t = 1 - 0.98^2 = 0.0396$ ，因此对第二天温度的估测变成了 $\frac{v_2}{0.0396} = \frac{0.0196\theta_1 + 0.02\theta_2}{0.0396}$ ，也就是 θ_1 和 θ_2 的加权平均数，并去除了偏差。你会发现随着 t 增加， β^t 接近于0，所以当 t 很大的时候，偏差修正几乎没有作用，因此当 t 较大的时候，紫线基本和绿线重合了。不过在开始学习阶段，你才开始预测热身练习，偏差修正可以帮助你更好预测温度，偏差修正可以帮助你使结果从紫线变成绿线。

在机器学习中，在计算指数加权平均数的大部分时候，大家不在乎执行偏差修正，因为大部分人宁愿熬过初始时期，拿到具有偏差的估测，然后继续计算下去。如果你关心初始时期的偏差，在刚开始计算指数加权移动平均数的时候，偏差修正能帮助你在早期获取更好的估测。

所以你学会了计算指数加权移动平均数，我们接着用它来构建更好的优化算法吧！

2.6 动量梯度下降法（Gradient descent with Momentum）

还有一种算法叫做**Momentum**，或者叫做动量梯度下降法，运行速度几乎总是快于标准的梯度下降算法，简而言之，基本的想法就是计算梯度的指数加权平均数，并利用该梯度更新你的权重，在本视频中，我们呢要一起拆解单句描述，看看你到底如何计算。

例如，如果你要优化成本函数，函数形状如图，红点代表最小值的位置，假设你从这里（蓝色点）开始梯度下降法，如果进行梯度下降法的一次迭代，无论是**batch**或**mini-batch**下降法，也许会指向这里，现在在椭圆的另一边，计算下一步梯度下降，结果或许如此，然后再计算一步，再一步，计算下去，你会发现梯度下降法要很多计算步骤对吧？

慢慢摆动到最小值，这种上下波动减慢了梯度下降法的速度，你就无法使用更大的学习率，如果你要用较大的学习率（紫色箭头），结果可能会偏离函数的范围，为了避免摆动过大，你要用一个较小的学习率。

另一个看待问题的角度是，在纵轴上，你希望学习慢一点，因为你不要这些摆动，但是在横轴上，你希望加快学习，你希望快速从左向右移，移向最小值，移向红点。所以使用动量梯度下降法，你需要做的是，在每次迭代中，确切来说在第 t 次迭代的过程中，你会计算微分 dW ， db ，我会省略上标 $[l]$ ，你用现有的**mini-batch**计算 dW ， db 。如果你用**batch**梯度下降法，现在的**mini-batch**就是全部的**batch**，对于**batch**梯度下降法的效果是一样的。如果现有的**mini-batch**就是整个训练集，效果也不错，你要做的是计算 $v_{dW} = \beta v_{dW} + (1 - \beta) dW$ ，这跟我们之前的计算相似，也就是 $v = \beta v + (1 - \beta) \theta_t$ ， dW 的移动平均数，接着同样地计算 v_{db} ， $v_{db} = \beta v_{db} + (1 - \beta) db$ ，然后重新赋值权重， $W := W - \alpha v_{dW}$ ，同样 $b := b - \alpha v_{db}$ ，这样就可以减缓梯度下降的幅度。

例如，在上几个导数中，你会发现这些纵轴上的摆动平均值接近于零，所以在纵轴方向，你希望放慢一点，平均过程中，正负数相互抵消，所以平均值接近于零。但在横轴方向，所有的微分都指向横轴方向，因此横轴方向的平均值仍然较大，因此用算法几次迭代后，你发现动量梯度下降法，最终纵轴方向的摆动变小了，横轴方向运动更快，因此你的算法走了一条更加直接的路径，在抵达最小值的路上减少了摆动。

动量梯度下降法的一个本质，这对有些人而不是所有人有效，就是如果你要最小化碗状函数，这是碗的形状，我画的不太好。

它们能够最小化碗状函数，这些微分项，想象它们为你从山上往下滚的一个球，提供了加速度，**Momentum**项相当于速度。

想象你有一个碗，你拿一个球，微分项给了这个球一个加速度，此时球正向山下滚，球因为加速度越滚越快，而因为 β 稍小于1，表现出一些摩擦力，所以球不会无限加速下去，所以不像梯度下降法，每一步都独立于之前的步骤，你的球可以向下滚，获得动量，可以从碗向下加速获得动量。我发现这个球从碗滚下的比喻，物理能力强的人接受得比较好，但不是所有人都能接受，如果球从碗中滚下这个比喻，你理解不了，别担心。

最后我们来看具体如何计算，算法在此。

所以你有两个超参数，学习率 α 以及参数 β ， β 控制着指数加权平均数。 β 最常用的值是0.9，我们之前平均了过去十天的温度，所以现在平均了前十次迭代的梯度。实际上 β 为0.9时，效果不错，你可以尝试不同的值，可以做一些超参数的研究，不过0.9是很棒的鲁棒数。那么关于偏差修正，所以你要拿 v_{dW} 和 v_{db} 除以 $1 - \beta^t$ ，实际上人们不这么做，因为10次迭代之后，因为你的移动平均已经过了初始阶段。实际中，在使用梯度下降法或动量梯度下降法时，人们不会受到偏差修正的困扰。当然 v_{dW} 初始值是0，要注意到这是和 dW 拥有相同维数的零矩阵，也就是跟 W 拥有相同的维数， v_{db} 的初始值也是向量零，所以和 db 拥有相同的维数，也就是和 b 是同一维数。

最后要说一点，如果你查阅了动量梯度下降法相关资料，你经常会看到一个被删除了的专业词汇， $1 - \beta$ 被删除了，最后得到的是 $v_{dW} = \beta v_{dW} + dW$ 。用紫色版本的结果就是，所以 v_{dW} 缩小了 $1 - \beta$ 倍，相当于乘以 $\frac{1}{1-\beta}$ ，所以你要用梯度下降最新值的话， α 要根据 $\frac{1}{1-\beta}$ 相应变化。实际上，二者效果都不错，只会影响到学习率 α 的最佳值。我觉得这个公式用起来没有那么自然，因为有一个影响，如果你最后要调整超参数 β ，就会影响到 v_{dW} 和 v_{db} ，你也许还要修改学习率 α ，所以我更喜欢左边的公式，而不是删去了 $1 - \beta$ 的这个公式，所以我更倾向于使用左边的公式，也就是有 $1 - \beta$ 的这个公式，但是两个公式都将 β 设置为0.9，是超参数的常见选择，只是在这两个公式中，学习率 α 的调整会有所不同。

所以这就是动量梯度下降法，这个算法肯定要好于没有**Momentum**的梯度下降算法，我们还可以做别的事情来加快学习算法，我们将在接下来的视频中探讨这些问题。

2.7 RMSprop

你们知道了动量（**Momentum**）可以加快梯度下降，还有一个叫做**RMSprop**的算法，全称是root mean square prop算法，它也可以加速梯度下降，我们来看看它是如何运作的。

回忆一下我们之前的例子，如果你执行梯度下降，虽然横轴方向正在推进，但纵轴方向会有大幅度摆动，为了分析这个例子，假设纵轴代表参数 b ，横轴代表参数 W ，可能有 W_1 ， W_2 或者其它重要的参数，为了便于理解，被称为 b 和 W 。

所以，你想减缓 b 方向的学习，即纵轴方向，同时加快，至少不是减缓横轴方向的学习，**RMSprop**算法可以实现这一点。

在第 t 次迭代中，该算法会照常计算当下mini-batch的微分 dW ， db ，所以我会保留这个指数加权平均数，我们用到新符号 S_{dW} ，而不是 v_{dW} ，因此 $S_{dW} = \beta S_{dW} + (1 - \beta)dW^2$ ，澄清一下，这个平方的操作是针对这一整个符号的，这样做能够保留微分平方的加权平均数，同样 $S_{db} = \beta S_{db} + (1 - \beta)db^2$ ，再说一次，平方是针对整个符号的操作。

接着**RMSprop**会这样更新参数值， $W := W - a \frac{dW}{\sqrt{S_{dW}}}$ ， $b := b - \alpha \frac{db}{\sqrt{S_{db}}}$ ，我们来理解一下其原理。记得在横轴方向或者在例子中的 W 方向，我们希望学习速度快，而在垂直方向，也就是例子中的 b 方向，我们希望减缓纵轴上的摆动，所以有了 S_{dW} 和 S_{db} ，我们希望 S_{dW} 会相对较小，所以我们要除以一个较小的数，而希望 S_{db} 又较大，所以这里我们要除以较大的数字，这样就可以减缓纵轴上的变化。你看这些微分，垂直方向的要比水平方向的大得多，所以斜率在 b 方向特别大，所以这些微分中， db 较大， dW 较小，因为函数的倾斜程度，在纵轴上，也就是 b 方向上要大于在横轴上，也就是 W 方向上。 db 的平方较大，所以 S_{db} 也会较大，而相比之下， dW 会小一些，亦或 dW 平方会小一些，因此 S_{dW} 会小一些，结果就是纵轴上的更新要被一个较大的数相除，就能消除摆动，而水平方向的更新则被较小的数相除。

RMSprop的影响就是你的更新最后会变成这样（绿色线），纵轴方向上摆动较小，而横轴方向继续推进。还有个影响就是，你可以用一个更大学习率 a ，然后加快学习，而无须在纵轴上垂直方向偏离。

要说明一点，我一直把纵轴和横轴方向分别称为 b 和 W ，只是为了方便展示而已。实际中，你会处于参数的高维度空间，所以需要消除摆动的垂直维度，你需要消除摆动，实际上是参数 W_1 ， W_2 等的合集，水平维度可能 W_3 ， W_4 等等，因此把 W 和 b 分开只是方便说明。实际中 dW 是一个高维度的参数向量， db 也是一个高维度参数向量，但是你的直觉是，在你要消除摆动的维度中，最终你要计算一个更大的和值，这个平方和微分的加权平均值，所以你最后去掉了那些有摆动的方向。所以这就是**RMSprop**，全称是均方根，因为你将微分进行平方，然后最后使用平方根。

最后再就这个算法说一些细节的东西，然后我们再继续。下一个视频中，我们会将**RMSprop**和**Momentum**结合起来，我们在**Momentum**中采用超参数 β ，为了避免混淆，我们现在不用 β ，而采用超参数 β_2 以保证在**Momentum**和**RMSprop**中采用同一超参数。要确保你的算法不会除以0，如果 S_{dW} 的平方根趋近于0怎么办？得到的答案就非常大，为了确保数值稳定，在实际操练的时候，你要在分母上加上一个很小很小的 ϵ ， ϵ 是多少没关系， 10^{-8} 是个不错的选择，这只是保证数值能稳定一些，无论什么原因，你都不会除以一个很小很小的数。所以**RMSprop**跟**Momentum**有很相似的一点，可以消除梯度下降中的摆动，包括mini-batch梯度下降，并允许你使用一个更大的学习率 a ，从而加快你的算法学习速度。

所以你学会了如何运用**RMSprop**，这是给学习算法加速的另一方法。关于**RMSprop**的一个有趣的事是，它首次提出并不是在学术研究论文中，而是在多年前**Jeff Hinton**在**Coursera**的课程上。我想**Coursera**并不是故意打算成为一个传播新兴的学术研究的平台，但是却达到了意想不到的效果。就是从**Coursera**课程开始，**RMSprop**开始被人们广为熟知，并且发展迅猛。

我们讲过了**Momentum**，我们讲了**RMSprop**，如果二者结合起来，你会得到一个更好的优化算法，在下个视频中我们再好好讲一讲为什么。

2.8 Adam 优化算法(Adam optimization algorithm)

在深度学习的历史上，包括许多知名研究者在内，提出了优化算法，并很好地解决了一些问题，但随后这些优化算法被指出并不能一般化，并不适用于多种神经网络，时间久了，深度学习圈子里的人开始多少有些质疑全新的优化算法，很多人都觉得动量（**Momentum**）梯度下降法很好用，很难再想出更好的优化算法。所以**RMSprop**以及**Adam**优化算法（**Adam**优化算法也是本视频的内容），就是少有的经受住人们考验的两种算法，已被证明适用于不同的深度学习结构，这个算法我会毫不犹豫地推荐给你，因为很多人都试过，并且用它很好地解决了许多问题。

Adam优化算法基本上就是将**Momentum**和**RMSprop**结合在一起，那么来看看如何使用**Adam**算法。

使用**Adam**算法，首先你要初始化， $v_{dW} = 0$, $S_{dW} = 0$, $v_{db} = 0$, $S_{db} = 0$ ，在第 t 次迭代中，你要计算微分，用当前的**mini-batch**计算 dW , db ，一般你会用**mini-batch**梯度下降法。接下来计算**Momentum**指数加权平均数，所以 $v_{dW} = \beta_1 v_{dW} + (1 - \beta_1) dW$ （使用 β_1 ，这样就不会跟超参数 β_2 混淆，因为后面**RMSprop**要用到 β_2 ），使用**Momentum**时我们肯定会用这个公式，但现在不叫它 β ，而叫它 β_1 。同样 $v_{db} = \beta_1 v_{db} + (1 - \beta_1) db$ 。

接着你用**RMSprop**进行更新，即用不同的超参数 β_2 ， $S_{dW} = \beta_2 S_{dW} + (1 - \beta_2)(dW)^2$ ，再说一次，这里是对整个微分 dW 进行平方处理， $S_{db} = \beta_2 S_{db} + (1 - \beta_2)(db)^2$ 。

相当于**Momentum**更新了超参数 β_1 ，**RMSprop**更新了超参数 β_2 。一般使用**Adam**算法的时候，要计算偏差修正， $v_{dW}^{\text{corrected}}$ ，修正也就是在偏差修正之后，

$$v_{dW}^{\text{corrected}} = \frac{v_{dW}}{1 - \beta_1^t},$$

$$\text{同样 } v_{db}^{\text{corrected}} = \frac{v_{db}}{1 - \beta_1^t},$$

$$S \text{ 也使用偏差修正，也就是 } S_{dW}^{\text{corrected}} = \frac{S_{dW}}{1 - \beta_2^t}, S_{db}^{\text{corrected}} = \frac{S_{db}}{1 - \beta_2^t}。$$

最后更新权重，所以 W 更新后是 $W := W - \frac{a v_{dW}^{\text{corrected}}}{\sqrt{S_{dW}^{\text{corrected}} + \epsilon}}$ （如果你只是用**Momentum**，使用 v_{dW} 或者修正后的 v_{dW} ，但现在我们加入了**RMSprop**的部分，所以我们要除以修正后 S_{dW} 的平方根加上 ϵ ）。

$$\text{根据类似的公式更新 } b \text{ 值， } b := b - \frac{a v_{db}^{\text{corrected}}}{\sqrt{S_{db}^{\text{corrected}} + \epsilon}}。$$

所以**Adam**算法结合了**Momentum**和**RMSprop**梯度下降法，并且是一种极其常用的学习算法，被证明能有效适用于不同神经网络，适用于广泛的结构。

本算法中有很多超参数，超参数学习率 a 很重要，也经常需要调试，你可以尝试一系列值，然后看哪个有效。 β_1 常用的缺省值为0.9，这是 dW 的移动平均数，也就是 dW 的加权平均数，这是**Momentum**涉及的项。至于超参数 β_2 ，**Adam**论文作者，也就是**Adam**算法的发明者，推荐使用0.999，这是在计算 $(dW)^2$ 以及 $(db)^2$ 的移动加权平均值，关于 ϵ 的选择其实没那么重要，**Adam**论文的作者建议 ϵ 为 10^{-8} ，但你并不需要设置它，因为它并不会影响算法表现。但是在使用**Adam**的时候，人们往往使用缺省值即可， β_1 ， β_2 和 ϵ 都是如此，我觉得没人会去调整 ϵ ，然后尝试不同的 a 值，看看哪个效果最好。你也可以调整 β_1 和 β_2 ，但我认识的业内人士很少这么干。

为什么这个算法叫做**Adam**？**Adam**代表的是**Adaptive Moment Estimation**， β_1 用于计算这个微分（ dW ），叫做第一矩， β_2 用来计算平方数的指数加权平均数（ $(dW)^2$ ），叫做第二矩，所以**Adam**的名字由此而来，但是大家都简称**Adam**权威算法。

顺便提一下，我有一个老朋友兼合作伙伴叫做**Adam Coates**。据我所知，他跟**Adam**算法没有任何关系，不过我觉得他偶尔会用到这个算法，不过有时有人会问我这个问题，我想你可能也有相同的疑惑。

这就是关于**Adam**优化算法的全部内容，有了它，你可以更加快速地训练神经网络，在结束本周课程之前，我们还要讲一下超参数调整，以及更好地理解神经网络的优化问题有哪些。下个视频中，我们将讲讲学习率衰减。

2.9 学习率衰减(Learning rate decay)

加快学习算法的一个办法就是随时间慢慢减少学习率，我们将之称为学习率衰减，我们来看看如何做到，首先通过一个例子看看，为什么要计算学习率衰减。

假设你要使用**mini-batch**梯度下降法，**mini-batch**数量不大，大概64或者128个样本，在迭代过程中会有噪音（蓝色线），下降朝向这里的最小值，但是不会精确地收敛，所以你的算法最后在附近摆动，并不会真正收敛，因为你用的 a 是固定值，不同的**mini-batch**中有噪音。

但要慢慢减少学习率 a 的话，在初期的时候， a 学习率还较大，你的学习还是相对较快，但随着 a 变小，你的步伐也会变慢变小，所以最后你的曲线（绿色线）会在最小值附近的一小块区域里摆动，而不是在训练过程中，大幅度在最小值附近摆动。

所以慢慢减少 a 的本质在于，在学习初期，你能承受较大的步伐，但当开始收敛的时候，小一些的学习率能让你步伐小一些。

你可以这样做到学习率衰减，记得一代要遍历一次数据，如果你有以下这样的训练集，

你应该拆分成不同的**mini-batch**，第一次遍历训练集叫做第一代。第二次就是第二代，依此类推，你可以将 a 学习率设为 $a = \frac{1}{1+\text{decayrate}*\text{epoch-num}} a_0$ （**decay-rate**称为衰减率，**epoch-num**为代数， a_0 为初始学习率），注意这个衰减率是另一个你需要调整的超参数。

这里有一个具体例子，如果你计算了几代，也就是遍历了几次，如果 a_0 为0.2，衰减率**decay-rate**为1，那么在第一代中， $a = \frac{1}{1+1} a_0 = 0.1$ ，这是在代入这个公式计算（ $a = \frac{1}{1+\text{decayrate}*\text{epoch-num}} a_0$ ），此时衰减率是1而代数是1。在第二代学习率为0.67，第三代变成0.5，第四代0.4等等，你可以自己多计算几个数据。要理解，作为代数函数，根据上述公式，你的学习率呈递减趋势。如果你想用学习率衰减，要做的是要去尝试不同的值，包括超参数 a_0 ，以及超参数衰退率，找到合适的值，除了这个学习率衰减的公式，人们还会用其它的公式。

比如，这个叫做指数衰减，其中 a 相当于一个小于1的值，如 $a = 0.95^{\text{epoch-num}} a_0$ ，所以你的学习率呈指数下降。

人们用到的其它公式有 $a = \frac{k}{\sqrt{\text{epoch-num}}} a_0$ 或者 $a = \frac{k}{\sqrt{t}} a_0$ （ t 为**mini-batch**的数字）。

有时人们也会用一个离散下降的学习率，也就是某个步骤有某个学习率，一会之后，学习率减少了一半，一会儿减少一半，一会儿又一半，这就是离散下降（**discrete stair cease**）的意思。

到现在，我们讲了一些公式，看学习率 a 究竟如何随时间变化。人们有时候还会做一件事，手动衰减。如果你一次只训练一个模型，如果你要花上数小时或数天来训练，有些人的确会这么做，看看自己的模型训练，耗上数日，然后他们觉得，学习速率变慢了，我把 a 调小一点。手动控制 a 当然有用，时复一时，日复一日地手动调整 a ，只有模型数量小的时候有用，但有时候人们也会这么做。

所以现在你有了多个选择来控制学习率 a 。你可能会想，好多超参数，究竟我应该做哪一个选择，我觉得，现在担心为时过早。下一周，我们会讲到，如何系统选择超参数。对我而言，学习率衰减并不是我尝试的要点，设定一个固定的 a ，然后好好调整，会有很大的影响，学习率衰减的确大有裨益，有时候可以加快训练，但它并不是我会率先尝试的内容，但下周我们将涉及超参数调整，你能学到更多系统的办法来管理所有的超参数，以及如何高效搜索超参数。

这就是学习率衰减，最后我还要讲讲神经网络中的局部最优以及鞍点，所以能更好理解在训练神经网络过程中，你的算法正在解决的优化问题，下个视频我们就好好聊聊这些问题。

2.10 局部最优的问题(The problem of local optima)

在深度学习研究早期，人们总是担心优化算法会困在极差的局部最优，不过随着深度学习理论不断发展，我们对局部最优的理解也发生了改变。我向你展示一下现在我们怎么看待局部最优以及深度学习中的优化问题。

这是曾经人们在想到局部最优时脑海里会出现的图，也许你想优化一些参数，我们把它们称之为 W_1 和 W_2 ，平面的高度就是损失函数。在图中似乎各处都分布着局部最优。梯度下降法或者某个算法可能困在一个局部最优中，而不会抵达全局最优。如果你要作图计算一个数字，比如说这两个维度，就容易出现有多个不同局部最优的图，而这些低维的图曾经影响了我们的理解，但是这些理解并不正确。事实上，如果你要创建一个神经网络，通常梯度为零的点并不是这个图中的局部最优点，实际上成本函数的零梯度点，通常是鞍点。

也就是在这个点，这里是 W_1 和 W_2 ，高度即成本函数 J 的值。

但是一个具有高维度空间的函数，如果梯度为0，那么在每个方向，它可能是凸函数，也可能是凹函数。如果你在2万维空间中，那么想要得到局部最优，所有的2万个方向都需要是这样，但发生的机率也许很小，也许是 2^{-20000} ，你更有可能遇到有些方向的曲线会这样向上弯曲，另一些方向曲线向下弯，而不是所有的都向上弯曲，因此在高维度空间，你更可能碰到鞍点。

就像下面的这种：

而不会碰到局部最优。至于为什么会把一个曲面叫做鞍点，你想象一下，就像是放在马背上的马鞍一样，如果这是马，这是马的头，这就是马的眼睛，画得不好请多包涵，然后你就是骑马的人，要坐在马鞍上，因此这里的这个点，导数为0的点，这个点叫做鞍点。我想那确实是你坐在马鞍上的那个点，而这里导数为0。

所以我们从深度学习历史中学到的一课就是，我们对低维度空间的大部分直觉，比如你可以画出上面的图，并不能应用到高维度空间中。适用于其它算法，因为如果你有2万个参数，那么 J 函数有2万个维度向量，你更可能遇到鞍点，而不是局部最优点。

如果局部最优不是问题，那么问题是什么？结果是平稳段会减缓学习，平稳段是一块区域，其中导数长时间接近于0，如果你在此处，梯度会从曲面从从上向下下降，因为梯度等于或接近0，曲面很平坦，你得花上很长时间慢慢抵达平稳段的这个点，因为左边或右边的随机扰动，我换个笔墨颜色，大家看得清楚一些，然后你的算法能够走出平稳段（红色笔）。

我们可以沿着这段长坡走，直到这里，然后走出平稳段。

所以此次视频的要点是，首先，你不太可能困在极差的局部最优中，条件是你训练较大的神经网络，存在大量参数，并且成本函数 J 被定义在较高的维度空间。

第二点，平稳段是一个问题，这样使得学习十分缓慢，这也是像**Momentum**或是**RMSprop**，**Adam**这样的算法，能够加速学习算法的地方。在这些情况下，更成熟的优化算法，如**Adam**算法，能够加快速度，让你尽早往下走出平稳段。

因为你的网络要解决优化问题，说实话，要面临如此之高的维度空间，我觉得没有人有那么好的直觉，知道这些空间长什么样，而且我们对它们的理解还在不断发展，不过我希望这一点能够让你更好地理解优化算法所面临的问题。